

Frame It as Cases Work That Speaks for Itself

Name: Jiya Yadav

Track: FlyRank AI Internship · Backend

Week: 2 Assignment A2

Voice Card

- Practical, Focused, Honest, Plainspoken, Authentic, Systematic, Uncompromising

Case 1 — FlyRank AI Week 2: REST API Under Constraints

The Problem

This assignment was not about building a todo app. It was about discipline under constraints.

The rules were clear:

- No database.
- In-memory only.
- Correct HTTP behavior.
- Proper validation.
- Clean documentation.

At first, it looked easy. It wasn't. The real challenge was enforcing correctness:

- Status codes must mean something.
- Validation must be predictable.
- The API contract must be clear without explanation.

On Windows, even testing became part of the problem. PowerShell altered JSON quoting before curl.exe received it. Errors looked wrong even when the API wasn't. That forced me to separate tooling issues from backend logic.

What I Did (and decided)

I kept everything in memory on purpose. Not because it was easier — but because adding a database would hide mistakes. With in-memory storage, every HTTP mistake is visible immediately.

Discipline decisions:

- Seeded 3 tasks → instant testability.
- Auto-increment IDs → predictable object lifecycle.
- Enforced correct status codes:
 - 200 read
 - 201 create
 - 204 delete
 - 404 unknown ID
 - 422 validation errors
- Allowed partial PUT updates → prevented unnecessary GET-before-PUT roundtrips.
- Added /stats → quick verification of state.
- Added /reset → test isolation without restarting the server.
- Added response_model=Task → Swagger stopped showing generic responses and became explicit.

I initially didn't use response_model. Swagger exposed that weakness immediately. I fixed it.

AI Rematch — What It Revealed

I rebuilt the project using AI from memory.

The differences were revealing:

- AI required a title in PUT → broke partial updates.
- AI converted all validation errors to 400.
- AI started with an empty dataset.

None of this meant my code was wrong. It showed something else:

- My prompt was vague.
- The AI made architectural decisions for me.

That was the lesson — ambiguity in prompts leads to unintended architecture.

What Came of It

This project sharpened three things:

1. I treat HTTP status codes as contracts, not suggestions.
2. I design APIs to reduce client friction (partial updates matter).
3. I separate tooling noise from backend truth.

The Windows curl.exe issue forced me to debug environment vs server behavior carefully.

It wasn't a big project.

But it forced precision.

And precision is the point.

About Me

I build backend systems with discipline under constraints.

I care about:

- Clear contracts
- Predictable behavior
- Testing without friction
- Decisions that can be defended

I don't add complexity unless it earns its place.

Contact / CTA

If you want someone who:

- Thinks in systems
- Explains tradeoffs
- Doesn't hide behind buzzwords

Let's talk.

Before / After Example

✗ Generic AI Line:

Built a scalable RESTful API with robust error handling and clean architecture.

✓ My Version:

Built an in-memory FastAPI service that enforces strict HTTP status codes, supports partial updates, and resets state without restarting the server.

One sounds like everyone.

One describes this project.